F1 Info & Tickets - recommender system

Razvoj softvera II · Aida Ušanović (IB220012)

1. Purpose

The mobile app recommends **upcoming races and a ticket type** to the signed-in customer. As described in the topic application, the recommendation takes into account previous race views, ticket purchases, favourite teams and drivers, track type, location and ticket class. Recommendations are shown on the mobile **Home** screen (*Recommended for you*), each with the reason and, when available, the ticket type the customer usually buys. They are served by `GET /api/Recommender`.

2. Input data

Input	Source	Used for
Purchased tickets (not refunded)	`Tickets`	user profile (weight 1.0 per race), excluding races the user already has tickets for, preferred ticket class
Race views	`RaceViews`, written by `POST /api/Race/{id}/view` when race details are opened	user profile (0.3 per view, at most 5 views per race)
Favourite drivers and teams	`FavoriteDrivers`, `FavoriteTeams`	favourite signal
Race results	`Results`	best finish of a favourite driver / team on each circuit
Track type, continent, country	`Circuits`, `Cities`, `Countries`	race feature vector
Ticket sales	`Tickets` of upcoming races	popularity signal

3. Signals and weights

Every upcoming race the user has no ticket for (one per circuit, the nearest edition) gets a score from three signals, each in the range 0..1:

Signal	How it is computed	Weight
Similarity	Cosine similarity between the race vector and the user profile (section 4).	0.60
Favourites	Best finish of a favourite driver, or of any driver of a favourite team, in earlier races on the same circuit: P1 = 1.0, P2 = 0.9 ... P10 = 0.1, outside the points = 0.	0.25
Popularity	Tickets sold for the race divided by the tickets sold for the best-selling upcoming race.	0.15

```
score = 0.60 · similarity + 0.25 · favourites + 0.15 · popularity
```

The five races with the highest score are returned (equal scores: the earlier race first). A user without history or favourites gets the most popular races; a user who only has favourites gets races on circuits where those drivers and teams did well.

4. User profile and race vector

The feature space has one position per track type, continent and country (read from the reference tables). A race vector sets its **track type = 1.0**, **continent = 1.0** and **country = 0.5**. The user profile is the weighted average of the vectors of the races in the user's history:

```
weight(race) = 1.0 · purchased + 0.3 · min(views, 5)
profile = Σ weight · vector(race) / Σ weight
similarity = cos(profile, vector(race)) = (p · r) / (|p| · |r|)
```

5. Reason shown with a recommendation

The reason is built from the **same weighted data used for ranking**: it describes the signal with the largest weighted contribution (0.60 · similarity, 0.25 · favourites or 0.15 · popularity) to that race's score.

- **Favourites**: "*Your favourite driver Max Verstappen won here in 2025.*" or "*Your favourite team Ferrari (Charles Leclerc) finished P2 here in 2026.*"
- **Similarity**: the track type and continent with the highest **weight** in the profile (purchases 1.0, views 0.3 each), e.g. "*You like races on "Street" circuits and races in Europe.*". Two purchased European races (weight 2.0) therefore outweigh three viewed Asian races (0.9), so the reason matches the ranking.
- **Popularity**: "*Popular with other fans (most tickets sold).*"

6. Ticket class

The ticket type the user bought most often (General Admission, Grandstand or Hospitality) is offered with every recommended race that still has seats of that type (`RecommendedTicketType` and `RecommendedTicketPrice` in the response). The race card shows "*Your usual ticket: Grandstand · €425.00*"; tapping it opens the ticket purchase with that type highlighted and preselected.

7. Main logic in the source code

Path: `F1Info&Tickets.Services/Services/RecommenderService.cs` (API: `F1Info&Tickets.WebAPI/Controllers/RecommenderController.cs`).

```
// Content-based recommendation of upcoming races. Every candidate race gets a score from three signals:
//  - similarity: cosine similarity between the race vector (track type, continent, country) and the user profile,
//    the weighted average of the races the user bought tickets for (1.0) or viewed (0.3 per view, at most 5 views);
//  - favourites: the best finish of a favourite driver, or of a driver of a favourite team, on the race's circuit;
//  - popularity: tickets sold for the race compared with the best-selling upcoming race.
// The reason shown with a race comes from the signal that contributed most to its score, and the ticket type
// the user buys most often is offered with every recommended race.
public class RecommenderService : IRecommenderService
{
    private const double PurchaseWeight = 1.0;
    private const double ViewWeight = 0.3;
    private const int MaxCountedViews = 5;

    private const double SimilarityWeight = 0.6;
    private const double FavoriteWeight = 0.25;
    private const double PopularityWeight = 0.15;

    // Only finishes in the points count for favourites: P1 = 1.0, P2 = 0.9 ... P10 = 0.1.
    private const int ScoredPositions = 10;
```

Signal weights.

```
public async Task<List<RaceResponse>> GetRecommendationsAsync(int userId, int topN = 5)
{
    var now = DateTime.UtcNow;
    var upcomingRaces = await _context.Races.IncludeRaceDetails()
        .Where(r => r.Date > now)
        .ToListAsync();
    if (!upcomingRaces.Any())
        return new List<RaceResponse>();

    var purchases = await _context.Tickets
        .Where(t => t.UserID == userId && t.Status != TicketStatus.Refunded)
        .Select(t => new { t.RaceTicketCategory.RaceID, t.RaceTicketCategory.TicketTypeID })
        .ToListAsync();
    var purchasedRaceIds = purchases.Select(p => p.RaceID).ToHashSet();

    var viewCounts = await _context.RaceViews
        .Where(v => v.UserID == userId)
        .GroupBy(v => v.RaceID)
        .Select(g => new { RaceID = g.Key, Count = g.Count() })
        .ToDictionaryAsync(x => x.RaceID, x => x.Count);

    var historyIds = purchasedRaceIds.Union(viewCounts.Keys).ToList();
    var historyRaces = await _context.Races.IncludeRaceDetails()
        .Where(r => historyIds.Contains(r.RaceID))
        .ToListAsync();

    var features = await BuildFeatureSpaceAsync();
    var profile = BuildProfile(historyRaces, race =>
        (purchasedRaceIds.Contains(race.RaceID) ? PurchaseWeight : 0)
        + ViewWeight * Math.Min(viewCounts.GetValueOrDefault(race.RaceID), MaxCountedViews), features);

    var favoriteFinishes = await LoadFavoriteFinishesAsync(userId);

    var popularity = await _context.Tickets
        .Where(t => t.Status != TicketStatus.Refunded && t.RaceTicketCategory.Race.Date > now)
        .GroupBy(t => t.RaceTicketCategory.RaceID)
        .Select(g => new { RaceID = g.Key, Count = g.Count() })
        .ToDictionaryAsync(x => x.RaceID, x => (double)x.Count);
    var maxPopularity = popularity.Values.DefaultIfEmpty(0).Max();
```

GetRecommendationsAsync, part 1: purchases, views, weighted profile, favourites and popularity.

```
    // One entry per circuit (the nearest edition), skipping races the user already has tickets for.
    var recommended = upcomingRaces
        .Where(r => !purchasedRaceIds.Contains(r.RaceID))
        .GroupBy(r => r.CircuitID)
        .Select(g => g.OrderBy(r => r.Date).First())
        .Select(race =>
        {
            var favorite = favoriteFinishes.GetValueOrDefault(race.CircuitID);
            return new ScoredRace(
                race,
                Similarity: profile == null ? 0 : SimilarityWeight * CosineSimilarity(profile.Vector, CreateRaceVector(race, features)),
                Favorite: FavoriteWeight * (favorite?.Score ?? 0),
                Popularity: maxPopularity > 0 ? PopularityWeight * popularity.GetValueOrDefault(race.RaceID) / maxPopularity : 0,
                favorite);
        })
        .OrderByDescending(x => x.Total)
        .ThenBy(x => x.Race.Date)
        .Take(topN)
        .ToList();

    var ticketOffers = await FindPreferredTicketOffersAsync(purchases.Select(p => p.TicketTypeID).ToList(), recommended);

    return recommended.Select(x =>
    {
        var response = _mapper.Map<RaceResponse>(x.Race);
        response.RecommendationReason = BuildReason(x, profile);
        if (ticketOffers.TryGetValue(x.Race.RaceID, out var offer))
        {
            response.RecommendedTicketTypeID = offer.TicketTypeID;
            response.RecommendedTicketType = offer.TicketType.Name;
            response.RecommendedTicketPrice = offer.Price;
        }
        return response;
    }).ToList();
}
```

GetRecommendationsAsync, part 2: candidates, weighted score, ranking, reason and ticket offer.

```
// Best finish (lowest position) of the user's favourite drivers and of drivers of favourite teams, per circuit.
private async Task<Dictionary<int, FavoriteFinish>> LoadFavoriteFinishesAsync(int userId)
{
    var driverIds = await _context.FavoriteDrivers.Where(f => f.UserID == userId).Select(f => f.DriverID).ToListAsync();
    var teamIds = await _context.FavoriteTeams.Where(f => f.UserID == userId).Select(f => f.TeamID).ToListAsync();
    if (!driverIds.Any() && !teamIds.Any())
        return new Dictionary<int, FavoriteFinish>();

    var finishes = await _context.Results
        .Where(r => r.Position <= ScoredPositions && (driverIds.Contains(r.DriverID) || teamIds.Contains(r.TeamID)))
        .Select(r => new
        {
            r.Race.CircuitID,
            r.Position,
            r.Race.Season.Year,
            r.DriverID,
            DriverName = r.Driver.FirstName + " " + r.Driver.LastName,
            r.Team.TeamName
        })
        .ToListAsync();

    return finishes
        .GroupBy(f => f.CircuitID)
        .ToDictionary(g => g.Key, g =>
        {
            // For an equal position the more recent season is the better story.
            var best = g.OrderBy(f => f.Position).ThenByDescending(f => f.Year).First();
            var who = driverIds.Contains(best.DriverID)
                ? $"Your favourite driver {best.DriverName}"
                : $"Your favourite team {best.TeamName} ({best.DriverName})";
            return new FavoriteFinish(who, best.Position, best.Year);
        });
}
```

Best finish of favourite drivers and teams per circuit (from the Results table).

```
// The ticket type the user bought most often, offered for each recommended race that still has seats of that type.
private async Task<Dictionary<int, RaceTicketCategory>> FindPreferredTicketOffersAsync(List<int> purchasedTicketTypeIds, List<ScoredRace> races)
{
    if (!purchasedTicketTypeIds.Any())
        return new Dictionary<int, RaceTicketCategory>();

    var preferredTypeId = purchasedTicketTypeIds
        .GroupBy(id => id)
        .OrderByDescending(g => g.Count())
        .ThenBy(g => g.Key)
        .First().Key;
    var raceIds = races.Select(r => r.Race.RaceID).ToList();

    var offers = await _context.RaceTicketCategories
        .Include(c => c.TicketType)
        .Where(c => raceIds.Contains(c.RaceID) && c.TicketTypeID == preferredTypeId)
        .Select(c => new { Category = c, Sold = c.Tickets.Count(t => t.Status != TicketStatus.Refunded) })
        .ToListAsync();

    return offers
        .Where(o => o.Sold < o.Category.Capacity)
        .ToDictionary(o => o.Category.RaceID, o => o.Category);
}
```

Ticket class: the most purchased ticket type, offered where seats are still available.

```
// Weighted average of the history race vectors, plus the track type and continent with the highest
// weight in it (used for the reason, so it always matches the ranking).
private record UserProfile(double[] Vector, string? TopTrackType, string? TopContinent);

private static UserProfile? BuildProfile(List<Race> history, Func<Race, double> weightOf, FeatureSpace features)
{
    var vector = new double[features.Count];
    var trackTypeWeights = new Dictionary<string, double>();
    var continentWeights = new Dictionary<string, double>();
    double totalWeight = 0;

    foreach (var race in history)
    {
        var weight = weightOf(race);
        if (weight <= 0)
            continue;
        var raceVector = CreateRaceVector(race, features);
        for (int i = 0; i < vector.Length; i++) vector[i] += weight * raceVector[i];
        totalWeight += weight;

        var trackType = race.Circuit.TrackType.Name;
        var continent = race.Circuit.City.Country.Continent.Name;
        trackTypeWeights[trackType] = trackTypeWeights.GetValueOrDefault(trackType) + weight;
        continentWeights[continent] = continentWeights.GetValueOrDefault(continent) + weight;
    }

    if (totalWeight == 0)
        return null;
    for (int i = 0; i < vector.Length; i++) vector[i] /= totalWeight;

    return new UserProfile(
        vector,
        trackTypeWeights.OrderByDescending(x => x.Value).First().Key,
        continentWeights.OrderByDescending(x => x.Value).First().Key);
}
```

Weighted user profile and its strongest track type and continent.

```
// The reason describes the signal with the largest weighted contribution to the race's score.
private static string BuildReason(ScoredRace scored, UserProfile? profile)
{
    var strongest = new[] { scored.Similarity, scored.Favorite, scored.Popularity }.Max();
    if (strongest <= 0)
        return "An upcoming race you have not attended yet.";

    if (scored.Favorite == strongest && scored.FavoriteFinish != null)
    {
        var finish = scored.FavoriteFinish;
        return finish.Position == 1
            ? $"{finish.Who} won here in {finish.Year}."
            : $"{finish.Who} finished P{finish.Position} here in {finish.Year}.";
    }

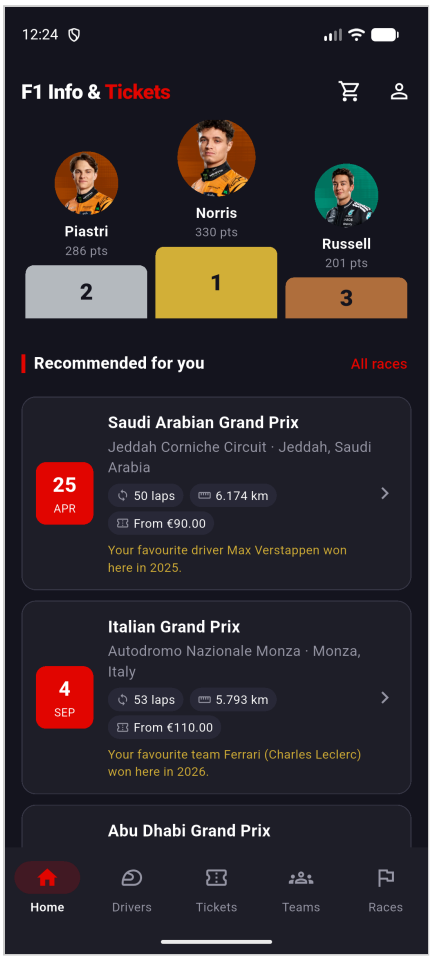
    if (scored.Similarity == strongest && profile != null)
    {
        var sameType = scored.Race.Circuit.TrackType.Name == profile.TopTrackType;
        var sameContinent = scored.Race.Circuit.City.Country.Continent.Name == profile.TopContinent;
        if (sameType && sameContinent)
            return $"You like races on \"{profile.TopTrackType}\" circuits and races in {profile.TopContinent}.";
        if (sameType)
            return $"Similar to the races you follow: a \"{profile.TopTrackType}\" circuit.";
        if (sameContinent)
            return $"A race in {profile.TopContinent}, the region you choose most often.";
        return "Similar to the races you viewed and bought tickets for.";
    }

    return "Popular with other fans (most tickets sold).";
}
```

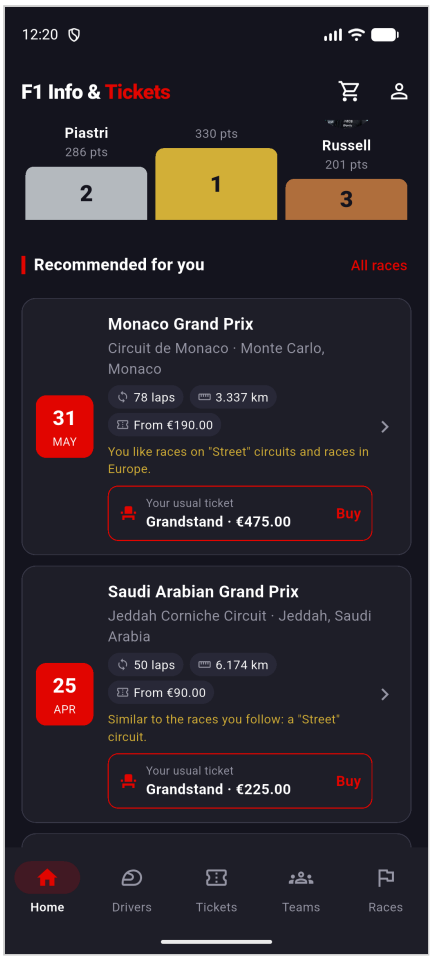
Reason from the signal with the largest weighted contribution.

8. Recommendations in the running application

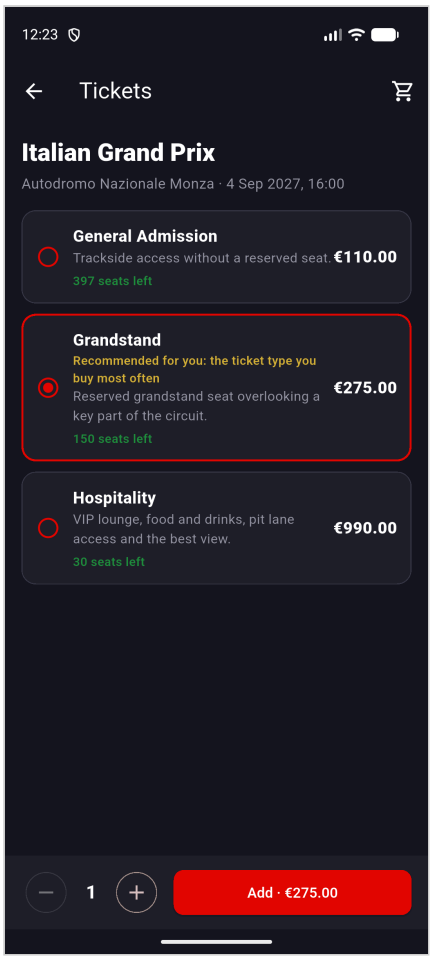
Path: `f1_info_and_tickets_frontend/lib/screens/mobile/home_tab.dart` (section *Recommended for you*; cards: `RaceCard` in `lib/widgets/mobile/mobile_widgets.dart`; ticket purchase with the recommended type: `lib/screens/mobile/race_tickets_screen.dart`).



User with favourite driver (Verstappen) and team (Ferrari): favourites signal and its reason.



User with purchase history: similarity reason and the ticket type they buy most often.



"Buy" opens the purchase with the recommended ticket type highlighted and preselected.